

UNAM

Universidad Nacional Autónoma de México

Facultad de Ciencias

Modelado y Programación

Práctica 02

Equipo Polimórfo

Juárez Larrondo Omar Alejandro
(322245244)

Soto Mendoza Ismael de Jesús
(322210422)

Vega Navas Saúl (322088267)

Identificación de la Práctica

Este documento corresponde a la **Práctica 02** de la materia *Modelado y Programación* de la Facultad de Ciencias, UNAM. El objetivo principal es implementar adecuadamente los patrones de diseño **State**, **Template Method** y **Decorator** en la resolución del problema propuesto para la cadena de pizzerías .^{El Pequeño Cesarín}.

1.1. Descripción del Problema

Se implementó un sistema robótico integral para resolver la escasez de personal en la pizzería .^{El Pequeño Cesarín}, donde un robot autónomo desempeña todas las funciones operativas de una sucursal completa. El robot opera mediante un flujo de trabajo estructurado que inicia con un estado de reposo hasta ser activado por la llegada de un cliente, procesa órdenes individuales que pueden incluir pizza, helado o ambos productos, confirma las órdenes antes de iniciar la preparación irreversible, ejecuta la preparación completa de los productos según recetas específicas, y finaliza con la entrega del pedido acompañado de un ticket detallado antes de retornar al estado de reposo.

El sistema incluye cinco variedades distintas de pizza que siguen un proceso de preparación unificado de once pasos secuenciales, donde cada variedad personaliza ingredientes específicos como tipos de queso y proteína según su receta particular. Las pizzas disponibles deben seleccionar entre tres tipos de masa diferentes proporcionando flexibilidad en la preparación según las preferencias del cliente. Adicionalmente, el sistema ofrece tres sabores de helado base que pueden ser personalizados dinámicamente mediante la adición de hasta ocho tipos diferentes de ingredientes extras, donde cada ingrediente puede ser agregado hasta un máximo de tres porciones y una vez añadido no puede ser removido, asegurando que el costo se ajuste progresivamente según las selecciones del cliente.

Análisis de Implementación de Patrones

2.1. Patrón State

¿Por qué se eligió esta implementación?

El patrón State fue implementado para gestionar los comportamientos complejos y dependientes del contexto tanto del robot como del sistema de pedidos, proporcionando una arquitectura que permite que los objetos alteren su comportamiento cuando su estado interno cambia. Esta elección se fundamenta en la necesidad de encapsular estados distintos con comportamientos específicos, eliminando las estructuras condicionales complejas que serían necesarias para manejar las múltiples transiciones de estado requeridas por el flujo operativo de la pizzería automatizada.

La implementación del patrón State se manifiesta en dos contextos principales dentro del sistema. El robot implementa cinco estados distintos que reflejan su ciclo operativo completo, iniciando con `EstadoDormido` donde permanece inactivo hasta ser despertado por un cliente, transitando a `EstadoTomandoOrden` cuando recibe la solicitud de atención, evolucionando a `EstadoConfirmarOrden` durante la validación del pedido, progresando a `EstadoTrabajando` durante la preparación de productos, y culminando en `EstadoEsperandoEntregar` hasta completar la entrega y retornar al estado inicial.

Paralelamente, el sistema de pedidos implementa una máquina de estados independiente que rastrea el progreso de cada pedido a través de cuatro estados secuenciales. El pedido inicia en `EstadoNoOrdenado` cuando se crea pero aún no se han agregado productos, progresa a `EstadoOrdenado` una vez que se confirma el pedido y se inicia la preparación, evoluciona a `EstadoHecho` cuando la preparación se completa exitosamente, y finaliza en `EstadoEntregado` después de la transferencia exitosa al cliente.

La separación de responsabilidades entre los estados del robot y del pedido permite que cada entidad mantenga su propia lógica de transición independiente, evitando el acoplamiento excesivo y proporcionando flexibilidad para modificaciones futuras. Esta implementación garantiza que cada estado encapsule tanto los comportamientos permitidos como las restricciones operativas, asegurando que acciones inapropiadas para el contexto actual sean rechazadas de manera consistente y predecible.

2.2. Patrón Template Method

¿Por qué se eligió esta implementación?

El patrón Template Method fue implementado para estandarizar el proceso de preparación de pizzas manteniendo la flexibilidad necesaria para personalizar ingredientes específicos según cada variedad. Esta elección se fundamenta en la necesidad de garantizar consistencia en los procedimientos operativos del robot mientras permite la diferenciación de productos, eliminando la duplicación de código que resultaría de implementar procesos de preparación completamente independientes para cada tipo de pizza.

La implementación se materializa a través de la clase abstracta `PreparadorPizza`, que define un algoritmo de preparación invariante compuesto por once pasos secuenciales que reflejan fielmente los requerimientos especificados para la operación de la pizzería. El método `template preparar()` establece la secuencia fija que incluye preparación de masa según el tipo seleccionado por el cliente, aplanado de la masa, colocación de salsa de tomate, aplicación de queso específico, agregado de especias aromáticas, incorporación de ingredientes especiales opcionales, adición de proteína específica cuando corresponda, introducción al horno de piedra, tiempo de espera estandarizado, extracción del horno y empaquetado final.

Las cinco implementaciones concretas del preparador personalizan aspectos específicos del algoritmo mediante la sobrescritura de métodos abstractos y hooks opcionales. `PreparadorPizzaMargherita` implementa la receta vegetariana clásica italiana sobrescribiendo el método `colocarQueso()` para utilizar mozzarella fresca di bufala y el hook `colocarIngredientesEspeciales()` para agregar albahaca fresca, omitiendo intencionalmente la proteína para mantener el carácter vegetariano. `PreparadorPizzaPepperoni` personaliza tanto el queso mediante mozzarella tradicional como la proteína especializándose en pepperoni italiano, mientras que `PreparadorPizzaHawaiana` combina mozzarella con jamón y piña como ingredientes distintivos.

La arquitectura permite que `PreparadorPizzaVegetariana` utilice una combinación de quesos vegetarianos y múltiples vegetales frescos sin proteína animal, mientras que `PreparadorPizzaCarneLover` maximiza el contenido proteico combinando múltiples tipos de carne con queso robusto. Esta implementación demuestra la versatilidad del patrón al permitir que cada preparador defina su identidad culinaria única dentro del marco estructural consistente, asegurando que el robot ejecute cada receta con la precisión y secuencia apropiadas independientemente de la complejidad específica de cada variedad.

2.3. Patrón Decorator

¿Por qué se eligió esta implementación?

El patrón Decorator fue implementado para gestionar el sistema de personalización de helados, proporcionando la flexibilidad necesaria para agregar dinámicamente ingredientes extras a un helado base sin modificar la estructura fundamental del producto. Esta elección se fundamenta en la necesidad de manejar combinaciones variables de ingredientes donde cada adición modifica tanto la descripción como el precio del producto final, evitando la explosión combinatoria de clases que resultaría de crear subclases específicas para cada combinación posible de ingredientes.

La implementación se estructura mediante la interfaz `ComponenteHelado` que define el contrato básico para todos los elementos del sistema, estableciendo métodos para obtener la descripción y el precio de cualquier componente helado. La clase `HeladoSimple` actúa como el componente concreto base, implementando un helado básico con uno de los tres sabores disponibles (fresa, vainilla o chocolate) y un precio fijo de 45 pesos, proporcionando la base sólida sobre la cual se pueden aplicar decoraciones progresivas.

La clase abstracta `DecoradorIngrediente` implementa la interfaz `ComponenteHelado` y mantiene una referencia al componente que decora, estableciendo la estructura fundamental que deben seguir todos los decoradores concretos de ingredientes. Esta arquitectura permite el encadenamiento de decoradores, donde cada ingrediente puede ser aplicado sobre un helado simple o sobre un helado ya decorado con

otros ingredientes, creando un sistema de composición flexible y extensible.

Los ocho decoradores concretos implementan ingredientes específicos con sus características particulares de costo y descripción. `ChispasChocolate` agrega chispas de chocolate con un costo de 12 pesos por porción, `GomitasGusano` incorpora gomitas de gusano a 8 pesos por porción, `Malvaviscos` añade malvaviscos a 10 pesos por porción, y así sucesivamente con `GomitasPanda`, `GomitasAro`, `Fresitas`, `Manguitos` y `Kiwis`, cada uno con su precio específico y capacidad de agregarse hasta tres veces por pedido.

La implementación permite que los clientes construyan helados personalizados mediante la aplicación secuencial de decoradores, donde cada ingrediente se apila sobre el anterior, modificando tanto la descripción textual como el precio acumulativo. Esta arquitectura satisface los requerimientos específicos del sistema donde los ingredientes no pueden ser removidos una vez agregados, el precio se ajusta dinámicamente con cada adición, y la descripción en el ticket refleja fielmente cada ingrediente aplicado, incluyendo repeticiones cuando se agregan múltiples porciones del mismo ingrediente.

Estado de la Implementación

3.1. Componentes Implementados

La práctica se encuentra **completamente implementada** con todos los componentes requeridos y funcionalidades especificadas en el enunciado original. El desarrollo incluye la correcta implementación de los tres patrones de diseño requeridos (State, Template Method y Decorator), la integración completa del sistema robótico con capacidades de gestión de pedidos multivariados, y un sistema de procesamiento de órdenes que maneja exitosamente tanto productos individuales como combinados.

El sistema incorpora cinco variedades distintas de pizza implementadas mediante el patrón Template Method, específicamente `Pizza Margherita` con mozzarella fresca di bufala y albahaca italiana, `Pizza Pepperoni` con mozzarella tradicional y pepperoni especiado, `Pizza Hawaiana` combinando mozzarella con jamón y piña tropical, `Pizza Vegetariana` utilizando quesos vegetarianos con vegetales frescos múltiples, y `Pizza Carne Lover` maximizando contenido proteico con múltiples carnes y queso robusto. Cada variedad implementa correctamente el proceso de preparación de once pasos con personalización específica de ingredientes según su receta característica.

El subsistema de helados implementa completamente el patrón Decorator mediante tres sabores base (fresa, vainilla y chocolate) que pueden ser decorados dinámicamente con ocho ingredientes diferentes. Los ingredientes disponibles incluyen chispas de chocolate, gomitas de gusano, gomitas de panda, gomitas de aro, malvaviscos, fresitas, manguitos y kiwis, donde cada ingrediente mantiene su precio específico y puede agregarse hasta tres porciones por helado según las especificaciones requeridas.

3.2. Estructura del Proyecto

El proyecto está organizado arquitectónicamente mediante una estructura de paquetes que refleja la separación clara de responsabilidades y facilita el mantenimiento del código. El paquete `main` contiene la clase principal `Main` que actúa como punto de entrada del sistema, mientras que el paquete `sistema` incluye las clases centrales `Robot`, `Sucursal`, `Pedido`, `Ticket`, `SistemaGestionPedidos` e `InterfazUsuarioCLI` que coordinan el funcionamiento integral del sistema.

La jerarquía de productos se estructura mediante el paquete `productos` que contiene la clase abstracta base `Producto` y las implementaciones concretas `Pizza` y `Helado`. El subpaquete `productos.preparadores` encapsula la implementación del patrón Template Method con `PreparadorPizza` como clase abstracta base y las cinco implementaciones concretas de preparadores específicos para cada variedad de pizza.

El sistema de helados se organiza dentro del subpaquete `productos.helados`, donde la interfaz `ComponenteHelado` define el contrato básico, `HeladoSimple` implementa los helados base, `DecoradorIngrediente` proporciona la estructura abstracta para decoradores, y las ocho clases concretas implementan cada ingrediente específico con sus características particulares de precio y descripción.

Los estados del sistema se organizan en dos paquetes especializados que reflejan la implementación dual del patrón State. El paquete `estado.robot` contiene la interfaz `EstadoActualRobot` y las cinco implementaciones concretas que gestionan el ciclo de vida operativo del robot, mientras que el paquete `estado.pedido` incluye `EstadoPedido` como interfaz base y las cuatro implementaciones que rastrean el progreso de cada pedido individual.

Finalmente, el paquete `enums` centraliza las enumeraciones `TipoMasa` y `SaborHelado` que proporcionan type safety y facilitan la extensibilidad futura del sistema, mientras que el paquete `sistema` adicional incluye utilidades como `GestorEntrada` para el manejo de entrada de datos del usuario, completando una arquitectura modular y bien estructurada.

Funcionamiento de la Práctica

El sistema opera mediante un flujo operativo integral que coordina meticulosamente la interacción entre el robot, los pedidos, los productos y la interfaz de usuario, implementando los tres patrones de diseño de manera armoniosa para proporcionar una experiencia de servicio completamente automatizada. La operación inicia con el robot en su estado dormido, representado por la clase `EstadoDormido`, donde permanece inactivo hasta recibir la señal de un cliente que requiere atención, momento en el cual el patrón State facilita la transición fluida hacia el estado activo de toma de órdenes.

Una vez despertado, el robot transiciona al `EstadoTomandoOrden`, donde se inicializa un nuevo pedido y se activa la interfaz de usuario mediante `InterfazUsuarioCLI` que presenta al cliente las opciones disponibles de productos. El sistema permite al cliente seleccionar entre pizzas, helados o una combinación de ambos productos, donde cada selección activa los mecanismos específicos de los patrones implementados según el tipo de producto elegido.

Cuando el cliente selecciona una pizza, el sistema presenta las cinco variedades disponibles y solicita la elección del tipo de masa entre las opciones napolitana, romana o americana. Una vez realizadas estas selecciones, el sistema crea una instancia de `Pizza` y asocia el preparador correspondiente según el tipo elegido, preparando la infraestructura necesaria para la posterior aplicación del patrón `Template Method` durante la fase de preparación. El producto se agrega al pedido actual y se actualiza el estado del pedido según corresponda.

Para las selecciones de helado, el sistema activa el mecanismo del patrón `Decorator` solicitando primero el sabor base entre fresa, vainilla y chocolate, creando una instancia de `HeladoSimple` con el sabor seleccionado. Posteriormente, el sistema presenta secuencialmente cada uno de los ocho ingredientes disponibles, permitiendo al cliente especificar la cantidad deseada de cada ingrediente hasta el máximo de tres porciones por tipo. Cada ingrediente seleccionado se implementa mediante un decorador específico que se aplica al helado base o al helado ya decorado, construyendo dinámicamente la configuración final del producto.

Una vez completada la selección de productos, el robot transiciona al `EstadoConfirmarOrden`, donde presenta al cliente un resumen completo del pedido incluyendo descripción y precios de todos los productos seleccionados. En este estado crítico, el cliente puede confirmar la orden para proceder con la preparación irreversible, o cancelar la orden completa, lo que resulta en la transición inmediata del robot de vuelta al `EstadoDormido` y la limpieza completa del pedido actual.

Cuando la orden se confirma exitosamente, el sistema marca el pedido como `EstadoOrdenado` y el robot transiciona al `EstadoTrabajando`, donde inicia el proceso de preparación de todos los productos del pedido. Para cada pizza en el pedido, el robot utiliza el preparador correspondiente que implementa el patrón `Template Method`, ejecutando secuencialmente los once pasos de preparación mientras muestra en tiempo real el progreso mediante mensajes descriptivos que detallan cada acción realizada según la receta específica de la pizza.

La preparación de helados sigue un proceso diferente pero igualmente estructurado, donde el robot solicita confirmación del sabor base y procede a agregar cada ingrediente decorador según la configuración

establecida durante la selección inicial. El sistema mantiene la consistencia del producto final aplicando cada decorador en el orden correcto y verificando que las cantidades correspondan exactamente con las especificaciones del pedido.

Una vez completada la preparación de todos los productos, el pedido transiciona al Estado `Hecho` y el robot evoluciona al Estado `EsperandoEntregar`, donde permanece hasta que el cliente solicite explícitamente la entrega de su orden. Durante este estado, el robot genera automáticamente un ticket detallado utilizando la clase `Ticket`, que recopila la descripción completa y el precio de cada producto, calculando el total final y incorporando la fecha y hora de generación.

El ticket resultante refleja fielmente la configuración de cada producto, mostrando el nombre de cada pizza sin detalles de ingredientes específicos según los requerimientos, y listando cada helado con su sabor base seguido de todos los ingredientes agregados, incluyendo repeticiones explícitas cuando se han agregado múltiples porciones del mismo ingrediente. Esta funcionalidad demuestra la integración exitosa del patrón `Decorator`, donde cada decorador contribuye apropiadamente tanto al precio como a la descripción final.

La entrega final se ejecuta cuando el cliente solicita su pedido, momento en el cual el robot presenta el ticket completo, entrega físicamente los productos preparados, marca el pedido como Estado `Entregado`, y transiciona automáticamente de vuelta al Estado `Dormido` para reiniciar el ciclo operativo. Este flujo completo demuestra la integración armoniosa de los tres patrones implementados, donde el patrón `State` gestiona las transiciones operativas, el patrón `Template Method` estandariza la preparación de pizzas, y el patrón `Decorator` proporciona flexibilidad en la personalización de helados, resultando en un sistema robusto y funcionalmente completo que satisface todos los requerimientos especificados.

Requisitos del Sistema

5.1. Versión de Java

La implementación de esta práctica requiere específicamente **Java Development Kit (JDK) versión 17** o superior para su correcta compilación y ejecución. Esta selección tecnológica se fundamenta en las características avanzadas y la estabilidad que proporciona Java 17 como versión de soporte a largo plazo (LTS), garantizando un entorno de desarrollo robusto y confiable para la implementación de patrones de diseño complejos y la gestión de sistemas orientados a objetos sofisticados.

Java 17 introduce optimizaciones significativas en el rendimiento de la máquina virtual que benefician directamente la ejecución de los patrones implementados en el sistema. Las mejoras en la gestión de memoria y el `garbage collection` optimizado facilitan el manejo eficiente de las múltiples instancias de objetos que se crean durante la aplicación del patrón `Decorator` en el sistema de helados, donde cada ingrediente adicional genera nuevos objetos decorador que se encadenan dinámicamente.

Las optimizaciones en el manejo de polimorfismo y `dispatch` de métodos virtuales mejoran significativamente el rendimiento de la implementación del patrón `State`, donde el robot y los pedidos cambian frecuentemente entre diferentes estados concretos, invocando métodos específicos según el estado actual. Estas mejoras son particularmente relevantes durante las transiciones de estado intensivas que ocurren durante el procesamiento de pedidos complejos que incluyen múltiples productos.

La versión seleccionada también proporciona mejor soporte para estructuras de datos complejas y manejo de colecciones, aspectos fundamentales para la gestión de listas de productos en pedidos, manejo de ingredientes múltiples en helados decorados, y coordinación de los diversos componentes del sistema durante la preparación y entrega de órdenes.

Adicionalmente, Java 17 incluye mejoras en el sistema de entrada y salida que optimizan la interacción con la interfaz de línea de comandos implementada en `InterfazUsuarioCLI`, proporcionando mejor responsividad durante la captura de entrada del usuario y la presentación de información del sistema. Estas características contribuyen a una experiencia de usuario más fluida durante la interacción con el sistema robótico automatizado.

Compilación y Ejecución del Proyecto

6.1. Proceso de Compilación

La compilación del proyecto se realiza mediante el compilador estándar de Java `javac`, aprovechando la estructura modular de paquetes que organiza las clases según sus responsabilidades funcionales específicas dentro del sistema de gestión de la pizzería automatizada. El proceso requiere la navegación al directorio raíz del proyecto, específicamente al directorio que contiene la carpeta `src`, desde donde se ejecutan las instrucciones de compilación que procesan recursivamente toda la jerarquía de clases del sistema.

La arquitectura del proyecto demanda una compilación ordenada que respete las dependencias entre las diferentes clases y paquetes, comenzando por las interfaces fundamentales y progresando hacia las implementaciones concretas que materializan los patrones de diseño implementados. El sistema de compilación debe procesar las interfaces `EstadoActualRobot`, `EstadoPedido` y `ComponenteHelado` antes de compilar sus implementaciones concretas, asegurando que todas las referencias se resuelvan correctamente durante el proceso de generación de bytecode.

Para efectuar la compilación completa del sistema, se debe posicionar en el directorio principal del proyecto y ejecutar la instrucción de compilación que localiza automáticamente todos los archivos fuente Java distribuidos en la estructura de paquetes. El comando específico para compilar todos los archivos del sistema es:

```
javac src/main/*.java src/main/**/*.java src/main/**/*.java
```

Este proceso se realiza mediante la aplicación sistemática del compilador `javac` a todos los archivos con extensión `.java` contenidos en el directorio `src` y sus subdirectorios correspondientes, respetando la jerarquía de paquetes establecida.

La compilación exitosa genera archivos de bytecode con extensión `.class` que mantienen la misma estructura jerárquica de directorios que el código fuente, permitiendo que la máquina virtual de Java localice y cargue correctamente todas las clases durante la ejecución posterior. Este proceso produce archivos optimizados que aprovechan las características específicas de Java 17 para proporcionar rendimiento óptimo durante la simulación completa del sistema.

6.2. Ejecución del Sistema

La ejecución del sistema se inicia mediante el comando estándar de Java `java`, especificando la clase principal `main.Main` que actúa como punto de entrada de toda la aplicación y coordina la inicialización completa del sistema de gestión de pedidos. La ejecución debe realizarse desde el directorio raíz del proyecto, donde se encuentran disponibles los archivos bytecode compilados organizados según la estructura de paquetes establecida durante la fase de desarrollo.

El comando de ejecución específico para iniciar el sistema completo utiliza el siguiente formato:

```
java -cp . main.Main
```

Este comando incorpora el parámetro `-cp .` que establece el classpath para incluir el directorio actual y todos sus subdirectorios en la búsqueda de clases, permitiendo que la máquina virtual localice apropiadamente todas las clases distribuidas en los diferentes paquetes del sistema. La instrucción activa la máquina virtual de Java, carga todas las clases necesarias según las dependencias del sistema, e inicia la ejecución secuencial del flujo operativo completo.

Durante la ejecución, el sistema presenta información detallada en la consola mostrando cada etapa del proceso operativo, incluyendo las transiciones de estado del robot, la aplicación de los patrones de

diseño durante la preparación de productos, y la generación de tickets finales. La salida en tiempo real permite supervisar el funcionamiento correcto de todos los patrones implementados, evidenciando las transiciones automáticas del patrón State, la ejecución secuencial del patrón Template Method durante la preparación de pizzas, y la aplicación dinámica del patrón Decorator durante la personalización de helados.

Para verificar que la compilación se ha realizado correctamente y confirmar el número total de archivos de clase generados durante el proceso, se puede utilizar el siguiente comando de verificación:

```
find . -name "*.class" | wc -l
```

Esta instrucción localiza recursivamente todos los archivos con extensión `.class` en el directorio actual y sus subdirectorios, contando automáticamente el total de archivos compilados mediante la combinación de los comandos `find` y `wc -l`. Este comando proporciona una verificación cuantitativa de la integridad del proceso de compilación, confirmando que todas las clases fuente han sido procesadas exitosamente y están disponibles para la ejecución del sistema.

El flujo completo de operación del sistema permite experimentar interactivamente con todos los aspectos implementados, desde la gestión de estados del robot hasta la personalización completa de productos mediante los patrones de diseño, proporcionando una demostración integral de la funcionalidad especificada para la pizzería .^{El} Pequeño Cesarín completamente automatizada.